

Cube配置

1. 基本配置(Systick+OLED)
 - 1-1 Cube配置
 - 1-2 基础代码
2. GPIO配置
 - 2-1 Cube配置
 - 2-2 基础代码
3. TIM中断定时
 - 3-0 **中断必看**
 - 3-1 Cube配置(TIM3)
 - 3-2 基础代码
4. TIM外部中断
 - 4-1 Cube配置
 - 4-2 基础代码
5. TIM输出比较(PWM)
 - 5-1 Cube配置(TIM3)
 - 5-2 基础代码
6. TIM输入捕获
 - 6-1 Cube配置(TIM5)
 - 6-2 基础代码
7. TIM编码器模式
 - 7-1 Cube配置
 - 7-2 基础代码
8. USART串口
 - 8-1 Cube配置
 - 8-2 基础代码
9. USART_DMA
 - 9-1 Cube配置
 - 9-2 基础代码

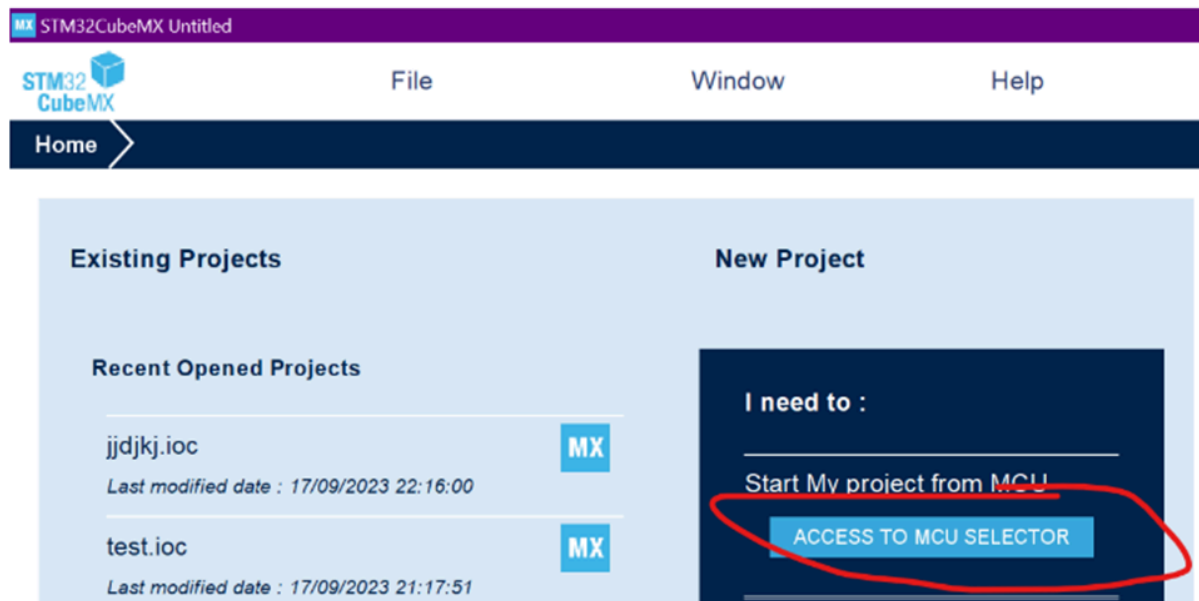
Cube配置

1. 基本配置(Systick+OLED)

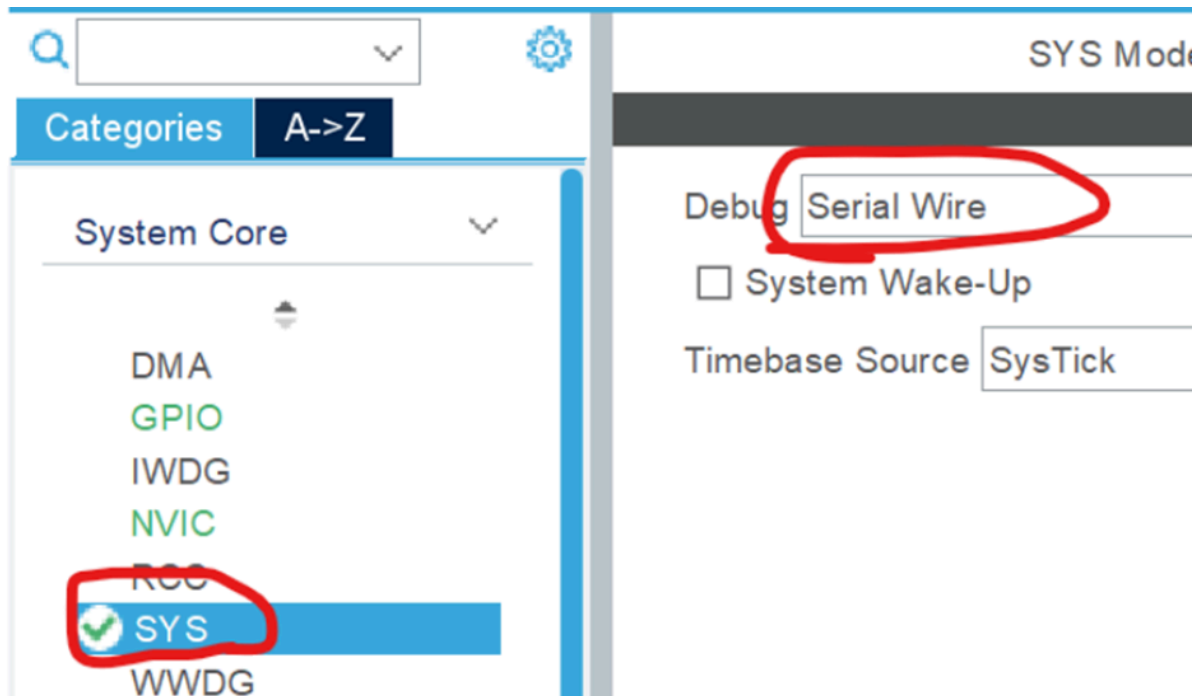
1-1 Cube配置

1. 进入页面(是MCU)

点击ACCESS TO MCU SELECTOR。



2. SYS配置



3. RCC配置

Clock Configuration

Pinout

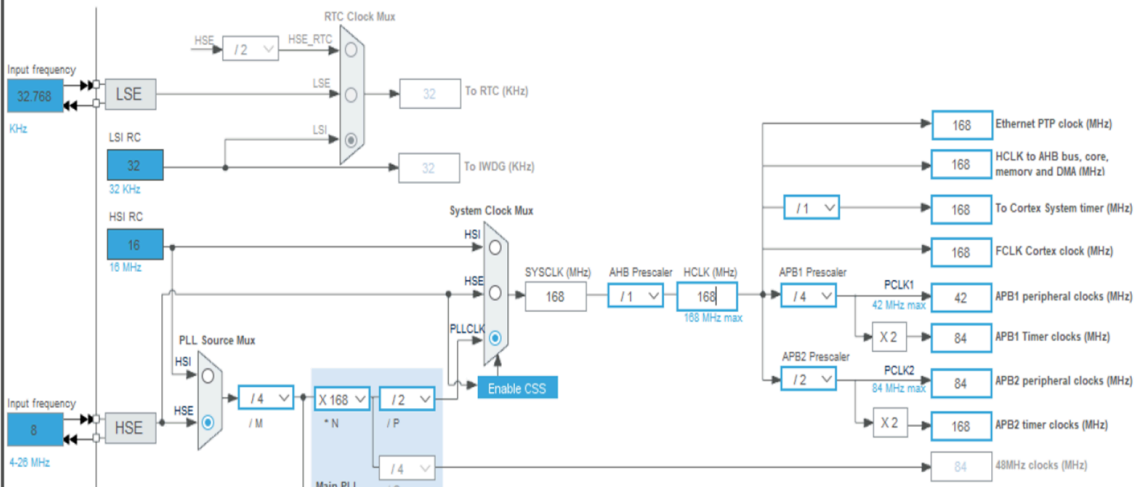
Mode

Crystal/Ceramic Resonator

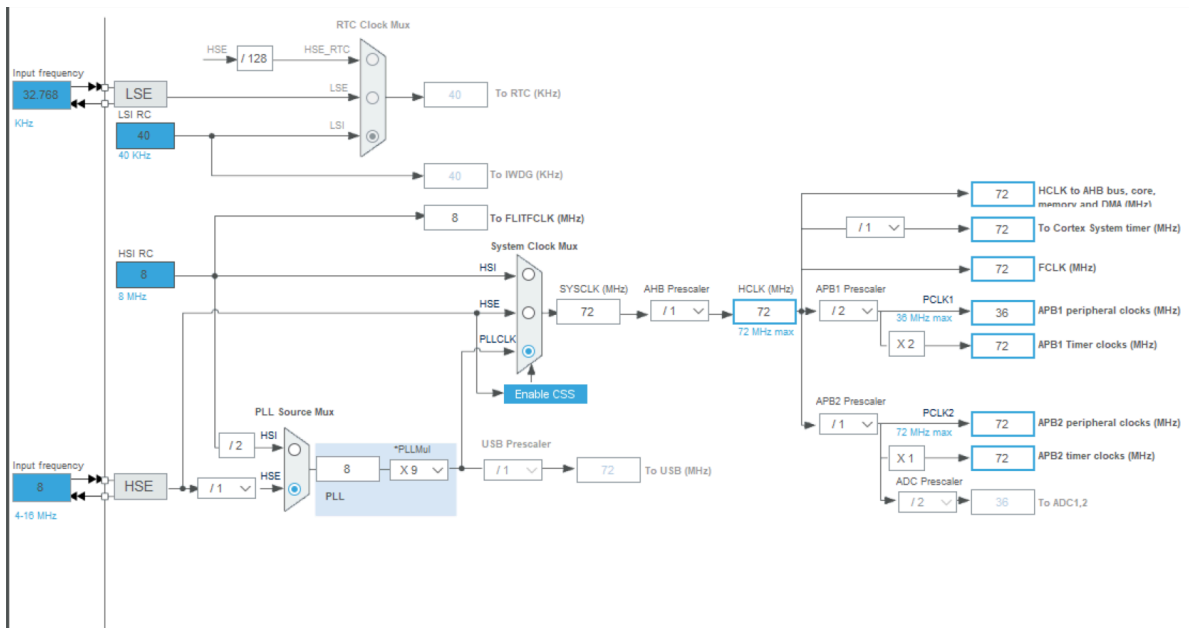
☐ Audio Clock Input (I2S_CKIN)

Time base: System tick timer	0	0
------------------------------	---	---

Resolve Clock Issues



注:STM32F103配置:



6. 工程配置---1

STM32CubeMX File Window Help Hello chen

Home > STM32F103C8Tx > Untitled - Project Manager > GENERATE CODE

Pinout & Configuration	Clock Configuration	Project Manager	Tools
Project	Project Name: test 工程名称	Project Location: E:\task\ 保存路径, 不能有中文	
Code Generator	Application Structure: Advanced ☐ Do not generate the main()	Toolchain Folder Location: E:\task\test\	
Advanced Settings	Toolchain / IDE: EWARM IDE选择, Keil选择MDK	Min Version: V8.32 ☐ Generate Under Root	
	Linker Settings		
	Thread-safe Settings		
	Cortex-M3NS		
	☐ Enable multi-threaded support Thread-safe Locking Strategy: Default - Mapping suitable strategy depending on RTOS selection.		
	Mcu and Firmware Package		
	Mcu Reference: STM32F103C8Tx		
	Firmware Package Name and Version: STM32Cube FW_F1 V1.8.5 ☒ Use latest available version		
	☒ Use Default Firmware Location		

7. 工程配置---2

Home > STM32F103C8Tx > Untitled - Project Manager > GENERATE CODE

Pinout & Configuration	Clock Configuration	Project Manager	Tools
Project	STM32Cube MCU packages and embedded software packs	☒ Copy all used libraries into the project folder 将所有使用的库复制到项目文件夹中 ☐ Copy only the necessary library files ☐ Add necessary library files as reference in the toolchain project configuration file	
Code Generator	Generated files	☒ Generate peripheral initialization as a pair of '.c/.h' files per peripheral 为每一个外设生成一对.c/.h文件 ☐ Backup previously generated files when re-generating ☒ Keep User Code when re-generating ☒ Delete previously generated files when not re-generated	
Advanced Settings	HAL Settings	☐ Set all free pins as analog (to optimize the power consumption) ☐ Enable Full Assert	
	User Actions		

8. OLED引脚配置

注:STM32F1与F4都是这个配置

✓ GPIO
✓ RCC
✓ SYS

Search Signals
☐ Show only Modified Pins

Pin Na...	Sig...	GPI...	GPIO mode	GPIO Pull-up/Pull-d...	Maxim...	User Label	Modified
PB8	n/a	Low	Output Push Pull	No pull-up and no p...	High	OLED_SCL	✓
PB9	n/a	Low	Output Push Pull	No pull-up and no p...	High	OLED_SDA	✓

PB9 Configuration :

GPIO output level

Low

GPIO mode

Output Push Pull

GPIO Pull-up/Pull-down

No pull-up and no pull-down

Maximum output speed

High

User Label

OLED_SDA

1-2 基础代码

1. 前提

- 主函数声明core_cm4.h
- main.h添加: `void HAL_SYSTICK_Callback(void)` ;
- stm32f4xx_it.c :找到函数SysTick_Handler,添加:HAL_SYSTICK_Callback() ;

2. setup

- `HAL_SYSTICK_Config(SystemCoreClock / 1000);` //0.001s定时周期(SystemCoreClock / x)

3. 回调函数

```
1 // 系统滴答定时器中断回调函数（每1ms调用一次）
2 void HAL_SYSTICK_Callback(void)
3 {
4     static volatile uint32_t ms_count = 0;
5
6     ms_count++; // 每毫秒递增计数器
7
8     // 检查是否达到1000ms（1秒）
9     if(ms_count >= 1000)
10    {
11        // 执行:LED状态翻转
12        HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);
13
14        ms_count = 0; // 重置计数器（避免溢出）
15    }
16 }
```

4. OLED代码:

```
1 // Setup:OLED初始化
2 OLED_Init() ;
3 OLED_ShowString(0,0,"Hello",OLED_8X16) ;
```

```
1 // while:OLED刷新
2 // OLED刷新(必要,放在最后)
3 OLED_Update() ;
```

2. GPIO配置

2-1 Cube配置

1. 输出模式(OUTPUT)

参数解释:

- GPIO output level : 引脚的初始状态(输出高电平or低电平),这里是给系统板载LED配置输出,根据原理图是引脚对VCC,所以初始为HIGH可以保证LED初始为暗
- GPIO mode : 推挽输出(PP)和开漏输出(OD)
- GPIO Pull-up / Pull-down : 选择上拉/下拉/浮空
- Maximun output speed : 输出速度,不了解,随便给呗
- User Label : 宏定义名字,建议给相关命名

Configuration

Group By Peripherals

GPIO

RCC

SYS

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pi

Pin N...	Signal on...	GPIO out...	GPIO mode	GPIO Pul...	Maximum...	User Label	Modifie
PG13	n/a	High	Output P...	No pull-u...	Low	LED0	☒

GPIO output level

High

GPIO mode

Output Push Pull

GPIO Pull-up/Pull-down

No pull-up and no pull-down

Maximum output speed

Low

User Label

LED0

2. 输入模式(Input)

参数解释:

见上面Output

GPIO Mode and Configuration

Configuration

Group By Peripherals

GPIO

RCC

SYS

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pin

Pin ...	Signal o...	GPIO ou...	GPIO m...	GPIO P...	Maximu...	User La...	Modifie
PF6	n/a	n/a	Input m...	Pull-up	n/a	KEY0	☒
PF7	n/a	n/a	Input m...	Pull-up	n/a	KEY1	☒
PF8	n/a	n/a	Input m...	Pull-up	n/a	KEY2	☒

PF8 Configuration :

GPIO mode

Input mode

GPIO Pull-up/Pull-down

Pull-up

User Label

KEY2

2-2 基础代码

省略

3. TIM中断定时

注:对于中断,F4的84M,F1为72M,所以使用F1时将84改为72,代码就差不多完全一样了

3-0 中断必看

- STM32F4定时器功能分组

通用定时器包括TIM2~ TIM5, TIM9~TIM14, 其中:

TIM2~ TIM5可以向上、向下和向上/向下计数, TIM9~TIM14只能向上计数。

TIM2~ TIM5支持编码器接口模式, TIM9~TIM14不支持。

TIM2~TIM5的捕获/比较通道有4路; TIM9和TIM12有2路; TIM10、TIM11 和 TIM13、TIM14只有1路。

TIM2~ TIM5 、TIM9和TIM12的时钟选择有多种; TIM10、TIM11 和 TIM13、TIM14的计数器时钟只能选择内部时钟;

- STM32F4定时器时钟频率

- 当APB1和APB2分频数为1的时候, TIM1、TIM8~TIM11的时钟为APB2的时钟, TIM2~TIM7、TIM12~TIM14的时钟为APB1的时钟;

2. 而如果APB1和APB2分频数不为1，那么TIM1、TIM8~TIM11的时钟为APB2的时钟的两倍，TIM2~TIM7、TIM12~TIM14的时钟为APB1的时钟的两倍。
3. 讲人话:由于在Cube中已经定义了时钟的分频(4),所以只看第2点就行了,下面是**定时器时钟频率**简易版:

APB1（时钟 84 MHz）,这些定时器的 定时器时钟频率 = 84 MHz。

TIM2 , TIM3 , TIM4 , TIM5 , TIM6 , TIM7 , TIM12 , TIM13 , TIM14

APB2（时钟 168 MHz）,这些定时器的 定时器时钟频率 = 168 MHz。

TIM1（高级定时器）,TIM8（高级定时器）,TIM9 ,TIM10 ,TIM11

- **STM32F1定时器功能分组**

类型	编号	总线	功能
高级定时器	TIM1、TIM8	APB2	拥有通用定时器全部功能，并额外具有重复计数器、死区生成、互补输出、刹车输入等功能
通用定时器	TIM2、TIM3、TIM4、TIM5	APB1	拥有基本定时器全部功能，并额外具有内外时钟源选择、输入捕获、输出比较、编码器接口、主从触发模式等功能
基本定时器	TIM6、TIM7	APB1	拥有定时中断、主模式触发DAC的功能

- STM32F103C8T6定时器资源：TIM1、TIM2、TIM3、TIM4

- **STM32F1定时器时钟频率**

- 都是72M

3-1 Cube配置(TIM3)

1. 参数配置

TIM3 Mode and Configuration

Mode

- Slave Mode: Disable
- Trigger Source: Disable
- Clock Source: Internal Clock

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

- Prescaler (PSC - 16 bits value): 8400-1
- Counter Mode: Up
- Counter Period (AutoReload Regi...): 5000-1
- Internal Clock Division (CKD): No Division
- auto-reload preload: Enable

Trigger Output (TRGO) Parameters

- Master/Slave Mode (MSM bit): Disable (Trigger input effect not delayed)
- Trigger Event Selection: Reset (UG bit from TIMx_EGR)

2. 打开NVIC中断(重点,不然没法定时)

Configuration

NVIC | Code generation

Priority Group: ... ☐ Sort by Preemption Priority and Sub Priority ☐ Sort by interrupts na

Search: Show: enabled interrupts ☒ Force DMA channel

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Prior
Non maskable interrupt	☒	0	0
Hard fault interrupt	☒	0	0
Memory management fault	☒	0	0
Pre-fetch fault, memory access fault	☒	0	0
Undefined instruction or illegal state	☒	0	0
System service call via SWI instruction	☒	0	0
Debug monitor	☒	0	0
Pendable request for system service	☒	0	0
Time base: System tick timer	☒	0	0
TIM3 global interrupt	☒	2	2

3-2 基础代码

1. 前提

```
1 // setup启动定时器中断模式
2 HAL_TIM_Base_Start_IT(&htim3);
```

2. 核心代码

```

1 // TIM3定时器
2 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
3 {
4     //判断是否是TIM3中断
5     if (htim->Instance == TIM3)
6     {
7         // 执行功能:
8         HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);
9     }
10 }

```

4. TIM外部中断

4-1 Cube配置

1. GPIO配置参数

- mode:可以选择上升沿or下降沿or双触发(中断)
- 上下拉自己选择,比如说这里就是选择上拉+下降沿触发+按键,所以按下按键触发中断
- Label记得取一个名字

PF6 Configuration :

GPIO mode	External Interrupt Mode with Falling edge trigger detection
GPIO Pull-up/Pull-down	Pull-up
User Label	Key_EXTI

2. 不要忘了开启外部中断哦

Configuration

Group By Peripherals

☒ GPIO
 ☒ RCC
 ☒ SYS
 ☒ TIM
 ☒ USART
 ☒ NVIC

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
EXTI line[9:5] interrupts	☒	0	0

4-2 基础代码

自己写回调函数

```

1 // 自己写回调函数
2 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
3 {
4     if (GPIO_Pin == Key_EXTI_Pin) // 判断引脚号
5     {
6         // 执行功能
7         check_1 ++ ;
8     }
9     // __HAL_GPIO_EXTI_CLEAR_FLAG(Key_EXTI_Pin) ; // 不需要重置标志位,HAL已经做好了
10 }

```

5. TIM输出比较(PWM)

5-1 Cube配置(TIM3)

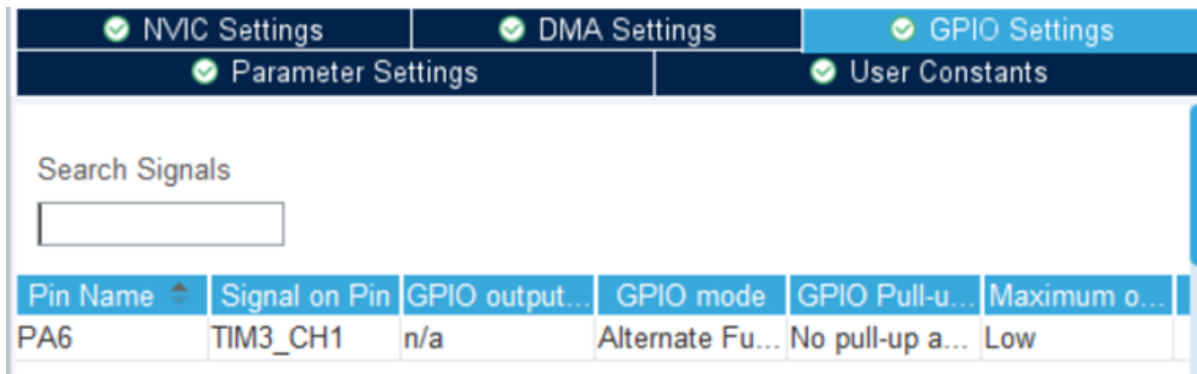
1. PWM配置

注:选择PWM模式1最好是把CH Polarity改为HIGH,这样CCRx越大PWM电平越大

- Clock Source: Internal Clock(选择内部时钟)
- Channel1:选择通道,一共四路,频率相同,对应四个GPIO,每个字都要对应哦
- PSC:记得小于65535
- ARR:CCRx<ARR则输出有效电平,CCRx>ARR则输出无效电平
- auto-reload : 自动重装,记得打开使能
- Mode:PWM1或者2
- CH Polarity:极性,建议**PWM1-HIGH**(这样CCRx越大,占空比越大)**PWM2-LOW**
- 不需要再NVIC使能中断,因为用不上

The screenshot shows the STM32CubeMX configuration tool for TIM3. The left sidebar lists various system components, with TIM3 selected under the 'Timers' category. The main configuration area is titled 'TIM3 Mode and Configuration'. It is divided into 'Mode' and 'Configuration' sections. In the 'Mode' section, 'Slave Mode' is set to 'Disable', 'Trigger Source' is 'Disable', 'Clock Source' is 'Internal Clock', and 'Channel1' is 'PWM Generation CH1'. The 'Configuration' section has a 'Reset Configuration' button and several tabs: 'Parameter Settings' (selected), 'User Constants', 'NVIC Settings', 'DMA Settings', and 'GPIO'. Under 'Parameter Settings', there are three expandable sections: 'Counter Settings' (Prescaler: 84-1, Counter Mode: Up, Counter Period: 500-1, Internal Clock Division: No Division, auto-reload preload: Enable), 'Trigger Output (TRGO) Parameters' (Master/Slave Mode: Disable, Trigger Event Selection: Reset), and 'PWM Generation Channel 1' (Mode: PWM mode 1, Pulse: 0, Output compare preload: Enable, Fast Mode: Disable, CH Polarity: Low).

2. 记一下引脚,没什么事



5-2 基础代码

1. 前提

```
1 // TIM3--PWM的初始化,选择四个通道中的通道1(注意要先在Cube中选择使能相应通道channel)  
2 HAL_TIM_PWM_Start(&htim3 , TIM_CHANNEL_1) ;
```

2. 核心代码

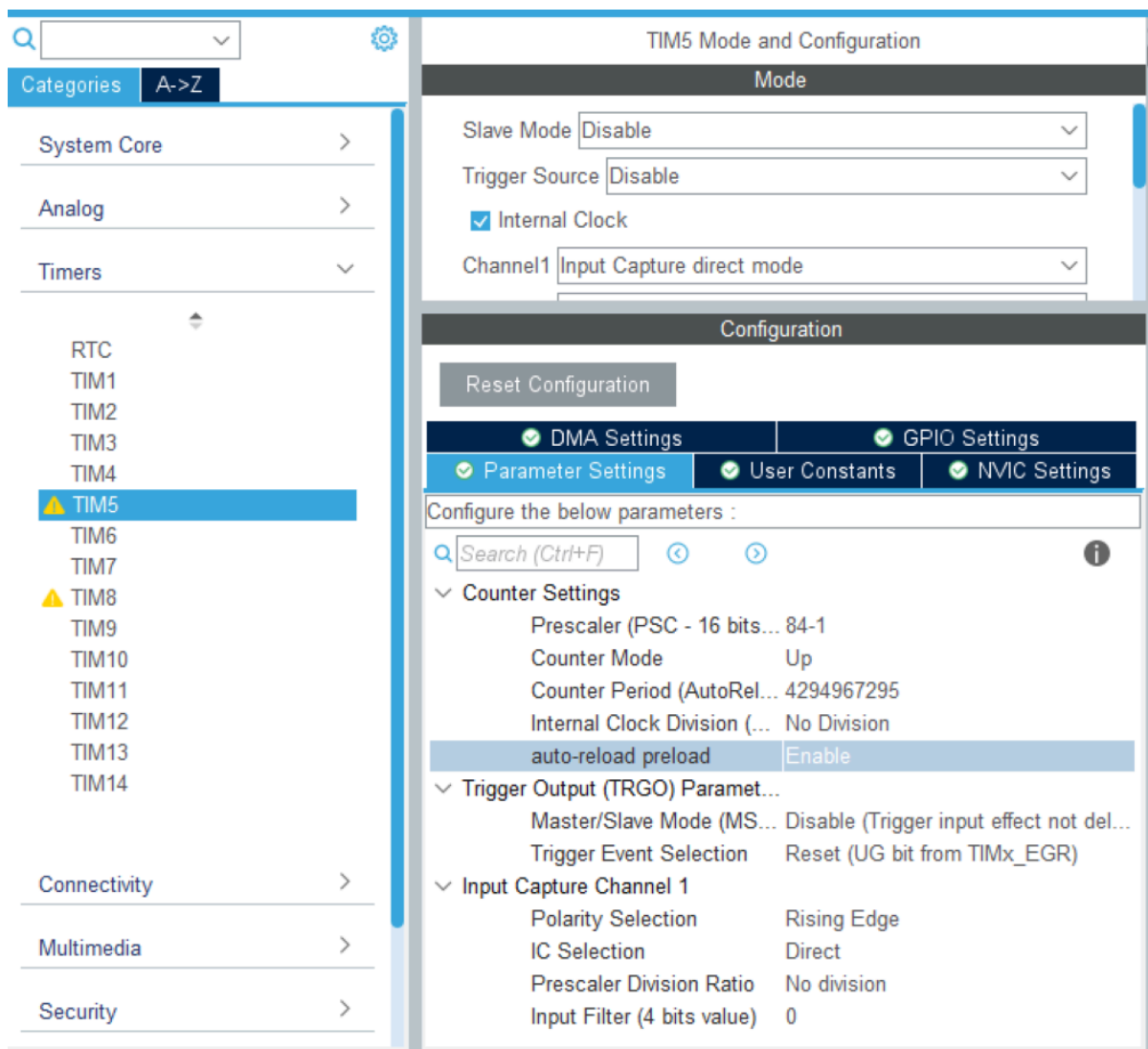
```
1 // 设置LED的占空比,LED_PWM需要在0-ARR之间  
2 __HAL_TIM_SET_COMPARE(&htim3 , TIM_CHANNEL_1 , LED_PWM ) ;
```

6. TIM输入捕获

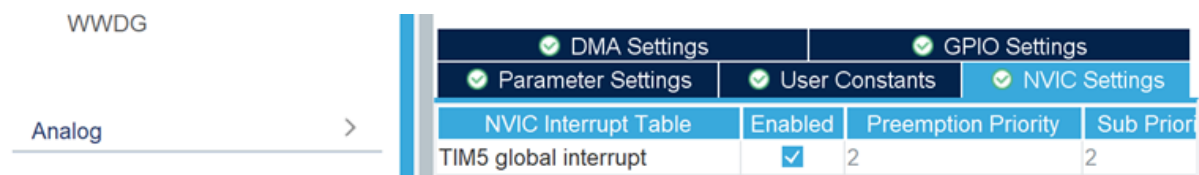
6-1 Cube配置(TIM5)

1. 参数配置

- 选择内部时钟
- channel1:输入捕获(IC)
- PSC没什么用,随便配置,给个84-1,PSC小.精度高,容易溢出,PSC大,精度低,可测量慢信号
- AAR设为最大,0xffffffff
- auto-reload preload(自动重载):使能(Enable)
- 选择捕获边缘:比如Rising Edge表示捕获上升沿



2. 开启中断



3. 看看引脚



6-2 基础代码

1. setup初始化

```

1 // TIM5初始化
2 HAL_TIM_Base_Start_IT(&htim5); // 启动更新中断
3 HAL_TIM_IC_Start_IT(&htim5, TIM_CHANNEL_1); // 启动输入捕获中断

```

2. 核心代码(捕获中断回调),以检测方波周期(上升沿到上升沿)为例

```

1 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
2 {
3     // 判断是否是TIM5且通道1引发的输入捕获中断
4     if (htim->Instance == TIM5 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
5     {
6         if (IsFirstCapture == 0)
7         {
8             // 记录得到第一次上升沿时间,单位为tick
9             IC_Val1 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
10            IsFirstCapture = 1;
11        }
12        else
13        {
14            // 第二次上升沿
15            IC_Val2 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
16
17            if (IC_Val2 > IC_Val1)
18                CapturePeriod = IC_Val2 - IC_Val1;
19            else
20                CapturePeriod = (0xFFFFFFFF - IC_Val1) + IC_Val2 + 1;
21
22            IsFirstCapture = 0;
23        }
24    }
25 }

```

7. TIM编码器模式

7-1 Cube配置

1. 参数配置

- 模式:Encoder Mode
- PSC = 0 , ARR = 65535(防止溢出)
- Encoder Mode : TI1 + TI2 (双边检测)
- Polarity:Rising(上升沿)
- Input Filter : 噪声滤波可以考虑,0-15为滤波标准

TIM3 Mode and Configuration

Mode

Slave Mode

Disable

Trigger Source

Disable

Clock Source

Disable

Channel1

Disable

Channel2

Disable

Channel3

Disable

Channel4

Disable

Combined Channels

Encoder Mode

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

⏪

⏩

ⓘ

Counter Settings

Prescaler (PSC - 16 bits value)

0

Counter Mode

Up

Counter Period (AutoReload Register - 16 bit...)

65535

Internal Clock Division (CKD)

No Division

auto-reload preload

Disable

Trigger Output (TRGO) Parameters

Encoder

Encoder Mode

Encoder Mode TI1 and TI2

Parameters for Channel 1

Polarity

Rising Edge

IC Selection

Direct

Prescaler Division Ratio

No division

Input Filter

0

Parameters for Channel 2

Polarity

Rising Edge

IC Selection

Direct

Prescaler Division Ratio

No division

Input Filter

0

7-2 基础代码

1. setup初始化

```
1 // 启动编码器接口
2 HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
```

2. 中断中计算转速

```
1 void HAL_SYSTICK_Callback(void)
2 {
3     // 启动自分频
4     static int ms_count = 0 ;
5     ms_count ++ ;
6
7     // 功能1: 编码器测速
```

```

8      if (ms_count == 1000)    // 采样间隔为1s
9      {
10         int16_t temp = __HAL_TIM_GET_COUNTER(&htim3) ;    // 得到一个采样周期期间
CNT的个数(不要溢出了否则CNT刷新,误差非常大)
11
12         speed = temp / cycle_CNT * 1 * 60 ;    // 1分钟的圈数(n/s) = 一个周期
(1s)的CNT值 / 一圈对应的CNT值 * 时间
13
14         __HAL_TIM_SET_COUNTER(&htim3 , 0) ; // 清零CNT,开启新的采样周期
15         ms_count = 1 ;
16     }
17 }

```

8. USART串口

8-1 Cube配置

1. 参数配置

- Mode:Asynchronous(异步通信)
- 波特率:115200

USART1 Mode and Configuration

Mode

Mode: Asynchronous

Hardware Flow Control (RS232): Disable

Configuration

Reset Configuration

☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings
 ☒ Parameter Settings
 ☒ User Constants

Configure the below parameters :

Search (Ctrl+F)

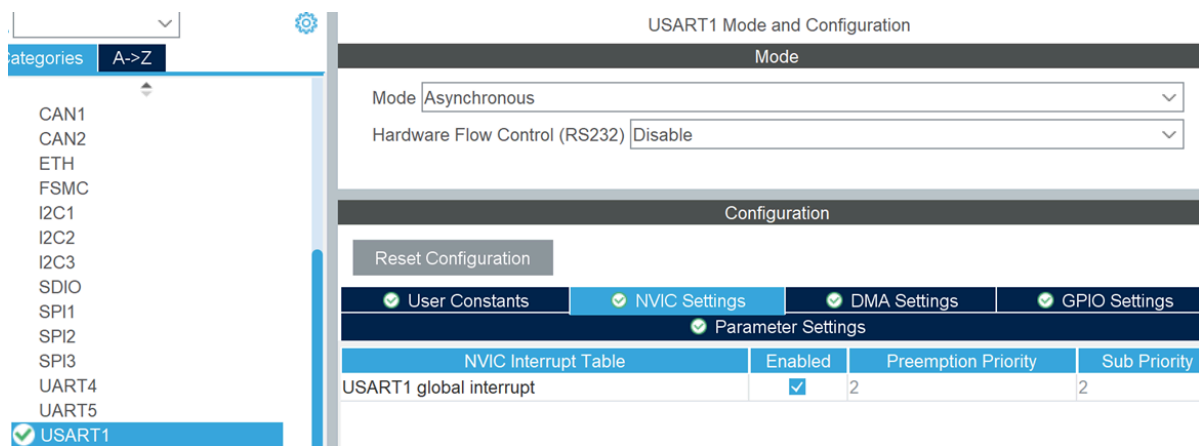
Basic Parameters

Baud Rate	115200 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1

Advanced Parameters

Data Direction	Receive and Transmit
Over Sampling	16 Samples

2. 中断配置



8-2 基础代码

1. 全局变量

```
1 // 定义接受长度
2 #define USART_RX_BUF_LEN 25
3 // 接收状态和缓冲区
4 uint8_t USART_RX_BUF[USART_RX_BUF_LEN] ;
```

2. setup初始化

```
1 // 接收中断初始化
2 HAL_UARTEx_ReceiveToIdle_IT(&huart1, USART_RX_BUF, USART_RX_BUF_LEN);
```

3. 接收函数中断初始化

```
1 // USART串口通信
2 void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
3 {
4     if (huart->Instance == USART1)
5     {
6         // 1.添加结束符确保字符串安全
7         if (Size > 0 && Size < USART_RX_BUF_LEN)
8         {
9             USART_RX_BUF[Size] = '\0';
10        }
11
12        // 2.使用字符串匹配代替固定长度检查
13        if (strstr((char *)USART_RX_BUF, "send") != NULL)
14        {
15            HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);
16        }
17
18        // 3. 发送回显（添加\r\n+正确长度+状态检查）
19        const char *echo_str = "Hello!"; // 不换行的回显字符串
20        uint16_t echo_len = strlen(echo_str); // 计算长度
21
22        if (huart1.gState == HAL_UART_STATE_READY)
23        {
24            HAL_UART_Transmit_IT(&huart1, (uint8_t *)echo_str, echo_len); //
25            检查UART是否就绪
26        }
27    }
28 }
```


Configuration

Reset Configuration

✓ NVIC Settings

✓ DMA Settings

✓ GPIO Settings

✓ Parameter Settings

✓ User Constants

DMA Request	Channel	Direction	Priority
USART1_RX	DMA1 Channel 5	Peripheral To Memory	Low
USART1_TX	DMA1 Channel 4	Memory To Peripheral	Low

Add

Delete

9-2 基础代码

- 全局变量

```
1 // DMA串口接收变量
2 #define RX_USART1_LEN 50
3 uint8_t RX_USART1[RX_USART1_LEN] ; // 接收数组
```

- setup初始化

```
1 HAL_UARTEx_ReceiveToIdle_DMA(&huart1 , RX_USART1 , RX_USART1_LEN ) ;// 开启
   DMA+接收空闲中断,接收的数据都存储在RX_USART1
```

- DMA发送数据

```
1 uint8_t sendbuff[] = "hello" ;
2 HAL_UART_Transmit_DMA(&huart1, (uint8_t *)sendbuff, sizeof(sendbuff));
```

- DMA接收中断

```
1 // 串口空闲中断回调函数
2 void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
3 {
4     if (huart->Instance == USART1)
5     {
6         // 加上\0防止越界
7         if (Size < RX_USART1_LEN)
8         {
9             RX_USART1[Size] = '\0';
10        }
11
12        // 清理缓冲区剩余数据,防止旧数据残留
13        memset(RX_USART1 + Size, 0, RX_USART1_LEN - Size);
14
15        // 发送回显
```

```
16     HAL_UART_Transmit_DMA(&huart1, (uint8_t *)RX_USART1,  
17     sizeof(RX_USART1));  
18     // 每次处理完需要重新开启DMA中断  
19     HAL_UARTEx_ReceiveToIdle_DMA(&huart1 , RX_USART1 , RX_USART1_LEN ) ;  
20 }  
21 }
```